

UNIVERSIDADE DA BEIRA INTERIOR
Covilhã | Portugal

Relatório do Trabalho Prático 1 - Farmácia UBI

Afonso Esteves - 55047
Tiago Pereira - 55019
Felipe Dourado - 53394

6 de novembro de 2025

Conteúdo

Resumo **3**

1 Introdução **4**
1.1 Âmbito, enquadramento e motivação 4

2 Contextualização do Projeto **4**
2.1 Identificação dos Desenvolvedores 4
2.1.1 Desenvolvedores Envolvidos 4
2.2 Contextualização Académica 4

3 Especificações Técnicas **5**
3.1 Ambiente de Desenvolvimento 5

4 Utilização **5**
4.1 Requisitos Prévios 5
4.2 Como executar o programa 5
4.3 Uso do Programa 6

5 Estrutura da Solução Construída **6**
5.1 Componentes do Sistema 6
5.2 Estrutura de Ficheiros do Projeto 7

6 Funcionalidades Implementadas **7**

7 Implementação de Funcionalidades **7**
7.1 Classe Farmacia: O Coração do Sistema 7
7.2 Classe Main: Interface com o Utilizador 8
7.3 Classe Encomenda: Gestão de Estados 9

8 Considerações de Implementação **10**
8.1 Princípios de Desenvolvimento 10
8.2 Desafios Encontrados 10

9 Conclusão **11**

10 Referências Bibliográficas **12**

Resumo

Este relatório detalha o desenvolvimento de uma aplicação de consola em Java para a gestão de uma farmácia. O sistema, desenvolvido no âmbito da disciplina de Programação Orientada a Objetos, permite gerir clientes, produtos e encomendas. A aplicação foi estruturada de forma a aplicar os conceitos fundamentais da POO, como encapsulamento, abstração e modelagem de entidades do mundo real em classes. O sistema oferece uma interface de linha de comandos para interagir com as funcionalidades implementadas.

Palavras-chave e *Keywords*

Java; Programação Orientada a Objetos; Sistema de Gestão; Farmácia; Consola.

1 Introdução

1.1 Âmbito, enquadramento e motivação

Este documento consiste num relatório técnico que descreve o desenvolvimento de uma aplicação de consola em Java, no contexto da unidade curricular de Programação Orientada a Objetos. A aplicação simula um sistema de gestão para uma farmácia, permitindo realizar operações básicas como o registo e a gestão de clientes, o controlo de stock de produtos e o processamento de encomendas.

A principal motivação para o desenvolvimento deste projeto foi a aplicação prática dos conceitos de Programação Orientada a Objetos (POO) aprendidos, traduzindo um problema do mundo real para um modelo de software coeso e funcional. O projeto permitiu explorar a modelação de entidades, a definição de responsabilidades entre classes e a criação de uma interface de utilizador textual.

O projeto situa-se na área de Desenvolvimento de Software, com foco na Programação Orientada a Objetos em Java e no desenvolvimento de aplicações de consola.

2 Contextualização do Projeto

2.1 Identificação dos Desenvolvedores

Os autores do presente projeto são estudantes do curso de Informática Web, Móvel e na Nuvem da Universidade da Beira Interior (UBI), matriculados na disciplina de Programação Orientada a Objetos.

2.1.1 Desenvolvedores Envolvidos

- **Afonso Esteves (55047)**
- **Felipe Dourado (53394)**
- **Tiago Pereira (55019)**

2.2 Contextualização Académica

O projeto foi desenvolvido no contexto da disciplina de Programação Orientada a Objetos, lecionada pelo Professor Paul Andrew Crocker, com o objetivo de aplicar e consolidar conhecimentos em:

- Modelação de problemas com base em objetos.
- Implementação de classes, atributos e métodos em Java.
- Utilização de conceitos como encapsulamento, construtores e relacionamentos entre classes.
- Gestão de coleções de objetos.
- Desenvolvimento de uma interface de utilizador baseada em texto.

3 Especificações Técnicas

3.1 Ambiente de Desenvolvimento

- **Linguagem:** Java
- **Ambiente de Execução:** Java Development Kit (JDK)
- **Ambiente de Desenvolvimento:** IntelliJ IDEA
- **Sistema Operacional:** macOS, Windows

4 Utilização

4.1 Requisitos Prévios

- **Java:** É necessário ter o JDK (Java Development Kit) instalado no sistema.

4.2 Como executar o programa

- **Compilação:**
  1. Navegue até à diretoria 'TP-01/src'.
  2. Compile os ficheiros Java com o seguinte comando:

     ```
     javac *.java
     ```

- **Execução:**
  1. Após a compilação, execute a aplicação com o comando:

```
java Main
```

- **Acesso:**
  - A aplicação será executada na consola, apresentando o menu principal.

4.3 Uso do Programa

- Ao iniciar a aplicação, será apresentado um menu principal com as opções de gestão de clientes, produtos e encomendas.
- O utilizador pode navegar pelos submenus para adicionar, remover, listar ou editar as informações.
- A interação é feita através da introdução de números correspondentes às opções de menu e do preenchimento dos dados solicitados.

5 Estrutura da Solução Construída

O sistema foi arquitetado com base nos princípios da Programação Orientada a Objetos, dividindo as responsabilidades em diferentes classes que representam as entidades do domínio do problema.

5.1 Componentes do Sistema

Classes de Modelo:

- `Cliente.java`: Representa um cliente da farmácia, com nome, NIF e morada.
- `Produto.java`: Representa um produto, com ID, nome, preço e stock.
- `Encomenda.java`: Modela uma encomenda, associando um cliente a um conjunto de produtos e registando o estado da mesma.
- `Morada.java`: Representa a morada de um cliente.
- `Farmacia.java`: Classe central que gere as listas de clientes, produtos e encomendas, e contém a lógica de negócio principal.
- `Main.java`: Ponto de entrada da aplicação, responsável pela interface com o utilizador (menus de consola).

5.2 Estrutura de Ficheiros do Projeto

```
TP-01/ .2 src/
    ├── Cliente.java
    ├── Encomenda.java
    ├── Farmacia.java
    ├── Main.java
    ├── Morada.java
    └── Produto.java
```

6 Funcionalidades Implementadas

As funcionalidades implementadas na aplicação incluem:

- **Gestão de Clientes:**
  - Adicionar, remover, listar e editar clientes.
  - Cada cliente é identificado por um NIF único.
- **Gestão de Produtos:**
  - Adicionar, remover, listar e editar produtos.
  - Controlo de stock (adicionar e remover unidades).
- **Gestão de Encomendas:**
  - Criar uma nova encomenda para um cliente, adicionando produtos do stock.
  - Listar todas as encomendas registadas.
  - Atualizar o estado de uma encomenda (ex: "Em processamento", "Pago", "Enviado", "Entregue").

7 Implementação de Funcionalidades

7.1 Classe Farmacia: O Coração do Sistema

A classe 'Farmacia' centraliza a lógica de negócio. Ela contém as listas de clientes, produtos e encomendas e os métodos para manipular esses dados.

```java
public void adicionarCliente(String nome, String nif, Morada
    morada) {
    if (acharClienteNif(nif) != null) {
        System.out.println("Erro:  J   existe um cliente com o NIF
            " + nif);
        return;
    }
    Cliente novoCliente = new Cliente(nome, nif, morada);
    this.clientes.add(novoCliente);
    System.out.println("Cliente adicionado com sucesso!");
}
```

Listing 1: Método para adicionar um cliente em Farmacia.java

7.2 Classe Main: Interface com o Utilizador

A classe ‘Main‘ é responsável por apresentar os menus ao utilizador e recolher o seu input, servindo de ponte para as funcionalidades da classe ‘Farmacia‘.

```java
public static void main(String[] args) {
    int opcao;
    do {
        System.out.println("\n--- BEM-VINDO    FARM CIA UBI ---");
        System.out.println("1. Gest o de Clientes");
        System.out.println("2. Gest o de Produtos");
        System.out.println("3. Gest o de Encomendas");
        System.out.println("0. Sair");
        System.out.print("\nEscolha uma  op   o: ");

        opcao = scanner.nextInt();
        scanner.nextLine();

       // Um switch para cada  op   o do menu.
        switch (opcao) {
            case 1:
                menuClientes();
                break;
            case 2:
                menuProdutos();
                break;
            case 3:
                menuEncomendas();
                break;
            case 0:
                System.out.println("Saindo do programa...");
                break;
            default:
                System.out.println("Op   o inv lida!");
        }
    } while (opcao != 0);

    scanner.close();
}
```

Listing 2: Menu principal em Main.java

7.3 Classe Encomenda: Gestão de Estados

A classe ‘Encomenda‘ utiliza um ‘enum‘ para gerir o estado do pedido, o que torna o código mais legível e seguro.

```java
public enum EstadoEncomenda {
    EM_PROCESSAMENTO("Em processamento"),
    PAGO("Pago"),
    ENVIADO("Enviado"),
    ENTREGUE("Entregue");

    private final String descricao;
    EstadoEncomenda(String descricao) {
        this.descricao = descricao;
    }
    public String getDescricao() {
        return descricao;
    }
}
```

Listing 3: Enum EstadoEncomenda em Encomenda.java

8 Considerações de Implementação

8.1 Princípios de Desenvolvimento

- **Separação de Responsabilidades:** A classe 'Main' trata da interação com o utilizador, enquanto a 'Farmacia' e as outras classes de modelo cuidam da lógica de negócio e dos dados.
- **Encapsulamento:** Os atributos das classes são privados e o acesso é feito através de métodos públicos (getters e setters), protegendo o estado interno dos objetos.
- **Validação de Dados:** Foram implementadas validações básicas para garantir a consistência dos dados, como a verificação de NIFs duplicados ou stocks negativos.

8.2 Desafios Encontrados

Durante o desenvolvimento do projeto, alguns desafios foram encontrados:

- **Gestão do Input do Utilizador:** Lidar com diferentes tipos de input (texto, números) e prevenir erros de execução devido a formatos inesperados exigiu atenção no uso da classe 'Scanner'.
- **Modelação do Relacionamento entre Classes:** Definir como as classes se relacionariam (ex: uma 'Encomenda' tem um 'Cliente' e

vários 'Produtos') foi um passo crucial para a correta estruturação do sistema.

- **Consistência dos Dados:** Garantir que o stock de um produto era atualizado corretamente ao criar uma encomenda foi um ponto importante para a integridade do sistema.

9 Conclusão

O desenvolvimento deste projeto permitiu aplicar de forma prática e consolidada os conceitos fundamentais da Programação Orientada a Objetos. A criação de um sistema de gestão de farmácia, mesmo que simplificado, demonstrou a eficácia da modelação de entidades do mundo real em classes e objetos para a construção de software modular e de fácil manutenção.

A experiência adquirida na estruturação das classes, na definição de responsabilidades e na implementação de uma interface de utilizador textual foi de grande valor para o aprofundamento dos conhecimentos em Java e no paradigma da POO.

10 Referências Bibliográficas

- Schildt, Herbert. *Java: A Beginner's Guide*. 6th ed., McGraw-Hill Education, 2014.
- Olsson, Mikael. *Java Quick Syntax Reference*. Apress, 2014.
- Roberts, Eric S. *The Art and Science of Java*. Addison-Wesley, 2008.

Acrónimos

UBI Universidade da Beira Interior